

Python module 01

python basics

- **Literals** are fixed values directly written in a program.
- They represent actual data (not variables).

Types of Literals

1. Numeric Literals

- **Integer** → whole numbers

Example:

```
a = 10
```

- **Float** → decimal numbers

Example:

```
b = 3.5
```

- **Complex** → numbers with real + imaginary part

Example:

```
c = 2 + 3j
```

2. String Literals

- Anything inside quotes is a string.
- Both single `' '` and double `" "` quotes work the same.

Example:

```
name = "Jashwanth"  
msg = 'Hello bro'
```

3. Boolean Literals

- Only two values:
 - `True`
 - `False`
- Must start with capital letters.

4. None Literal

- Represents no value / empty value.

Example:

```
x = None
```

String Rules

- Strings must start and end with the same **quote**.
- Valid:

```
"hello"  
'hello'
```

- Invalid:

```
"'hello'
```

Triple Quotes (Multi-line Strings)

- Used for writing multiple lines of text.

Example:

```
msg = """Hello  
Linux"""
```

Print Function

- Used to display output.

Example:

```
print("hello linux")
```

Output:

```
hello linux
```

Parentheses

- Parentheses `()` are used in functions.

Example:

```
print("Hello")
```

Key Concept

- **Literal** = actual value
- **Variable** = name that stores the value

Example:

```
x = 10
```

- `10` → literal
- `x` → variable

Types of Variables in Python

```
a = 10    # int (integer)
b = 3.5   # float (decimal)
c = "hello" # string (text)
d = True  # boolean (True/False)
```

Rules for Naming Variables

✓ Allowed:

```
name = "Jash"
_age = 20
marks1 = 95
```

✗ Not allowed:

```
1name = "wrong" # can't start with number
my-name = "wrong" # no hyphen
```

KEYWORDS

- 👉 **Keywords are special words**
- 👉 Python already understands them
- 👉 You **CANNOT** use them as variable names

Simple Meaning of Each (Easy Words)

- `if` → check condition
- `else` → otherwise
- `for` → repeat loop
- `while` → repeat while condition is true
- `break` → stop loop
- `def` → create function
- `return` → give output from function
- `True / False` → boolean values
- `and / or` → combine conditions

```
>> import keyword
>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
, 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
>>
```

Primitive Datatypes

type : function to know the variable data type

```
# Integers

age = 25
age
25

type(age)
<class 'int'>

# Floats

percent = 87.5
percent
87.5

type(percent)
<class 'float'>

# None

val3 = None
val3

\
\

type(val3)
<class 'NoneType'>
```

```
# Strings

name = "Mark"
name
'Mark'

type(name)
<class 'str'>

full_name = "Mark Wood"

full_name
'Mark Wood'

type(full_name)
<class 'str'>

# Boolean

# True and False

val1 = True

val1
True

type(val1)
<class 'bool'>
```

Basic operations with strings

```
>>> s1 = "Hello World"
>>> s1
'Hello World'
>>>
>>> type(s1)
<class 'str'>
>>>
>>> s2 = 'We are learning Python'
>>>
>>> s2
'We are learning Python'
>>>
>>> type(s2)
<class 'str'>
>>>
>>> s3 = """Hello everyone.
... We are looking at strings.
... Bye"""
>>>
>>> s3
'Hello everyone.\nWe are looking at strings.\nBye'
>>>
>>> type(s3)
<class 'str'>
```

`\n` represents new line

What is len()?

👉 `len()` is used to **count length (number of items)**

```
>>>
>>> s4 = "Python"
>>>
>>> len(s4)
6
>>>
>>> s1
'Hello World'
>>> len(s1)
11
```

in hello world 11 characters as 10 words and 1 space [space is also a valid character in python]

What is Indexing?

👉 Indexing = position of each item

Think like:

```
H E L L O  
0 1 2 3 4
```

👉 Each letter has a **number (index)** starting from **0**

 Example with String

```
s4 = "Python"  
  
len(s4)  
6  
  
s1  
'Hello World'  
len(s1)  
11  
  
s4[0]  
'P'  
  
s4[1]  
'y'  
s4[2]  
't'  
s4[5]  
'n'  
  
s4[6]  
Traceback (most recent call last):  
  File "<pyshell#33>", line 1, in <module>  
    s4[6]  
IndexError: string index out of range
```

What is Negative Indexing?

👉 It means counting from the end (right side)

Example

H E L L O

0 1 2 3 4 → normal indexing

-5 -4 -3 -2 -1 → negative indexing

```
s4= "python"
s4[-1]
'n'
s4[1]
'y'
s4[0]
'p'
s4[-2]
'o'
|
```

CONCATENATION OF STRINGS / *joining of string*

```
>>> s1 = 'hello'
>>> s2 = 'python'
>>> s1 + s2
'hellopython'
>>> s1 + ' ' + s2
'hello python'
```

not all types get add + eg string with float or integer with string

```
>>> language = "Python"
>>> version = 3.13
>>> # Python3.13
>>> language + version
Traceback (most recent call last):
  File "<pyshell#93>", line 1, in <module>
    language + version
TypeError: can only concatenate str (not "float") to str
```

→ str
→ float
str + float X

for this issue we need to use type casting or type converting for adding which we are gonna see next

Type conversion / Type casting

Integer = int , string = str , float = float , boolean = bool

```
>>> # Type conversion / Type casting
>>> num1 = 100
>>> type(num1)
<class 'int'>
>>> float(num1)
100.0
>>> num2 = float(num1)
>>> type(num2)
<class 'float'>
>>> type(num1)
<class 'int'>
>>> num1 = float(num1)
>>> num1
100.0
>>> type(num1)
<class 'float'>
```

type casting wont change the data type of previously assigned value untill unless we change it

```
>>> num3 = 56.45
>>> int(num3)
56
>>> int(77.99)
77
```

no rounding of float take place when we change it into integer

```
int(num3)
56
int(77.99)
77
str(num3)
'56.45'
str(num1)
'100.0'
```

limitations too not every data type converts into other data type :

```
>>> s1 = 'hello'
>>> int(s1)
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    int(s1)
ValueError: invalid literal for int() with base 10: 'hello'
>>> float(s1)
Traceback (most recent call last):
  File "<pyshell#18>", line 1, in <module>
    float(s1)
```

for coverting into integers or floats from strings that string shouldnt have any special characters such as letters or any others only numbers are required ...

issue in catenation problem

```
>>> langugae = 'python'
>>> version = 3.13
>>> langugae + version
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    langugae + version
TypeError: can only concatenate str (not "float") to str
>>> langugae + str(version)
'python3.13'
>>> langugae + ' ' + str(version)
'python 3.13'
>>>
```

Adding of strings remember :

Adding of integers

```
>>> '100' + '100'
'100100'
```

```
>>> int('100') + int('100')
200
```

```
>>> vall = True
>>> str(vall)
'True'
>>> bool(False)
False
>>> bool(false)
Traceback (most recent call last):
  File "<pyshell#13>", line 1, in <module>
    bool(false)
NameError: name 'false' is not defined. Did you mean: 'False'?
>>> bool(True)
True
>>> bool(true)
Traceback (most recent call last):
  File "<pyshell#15>", line 1, in <module>
    bool(true)
NameError: name 'true' is not defined. Did you mean: 'True'?
>>> bool(0)
False
>>> bool(1)
True
>>> bool('python')
True
>>> bool('100')
True
>>> bool(120001020102)
True
>>> bool(-0)
False
>>> bool(-1)
True
```

both positive and negative integers and floats gives true but 0 and 0.00 will give false

empty strings get false "" , "" , " no space

```
>>> bool(0.0)
False
>>>
>>>
>>> bool(0.5)
True
>>>
>>> bool("hi")
True
>>>
>>> bool('a')
True
>>>
>>> bool(' ')
True
>>>
>>> # Empty string => '', "", '','', """"
>>>
>>> bool("")
False
```

```
>>> # None
>>>
>>> bool(None)
False
```

```
int(True)
1

int(False)
0

float(True)
1.0

float(False)
0.0

str(True)
'True'

str(False)
'False'

|
str(None)
'None'
```

Arithmetic & Assignment Operators

y, x = variables

20 = value [both variables and value together are called as operands , + - / * are called operations]

```
>>> x = 10
>>> y = x + 20
>>> y
30
```

```
>>> # Arithmetic Operator
>>>
>>> num1 = 10
>>> num2 = 5
>>>
>>> num1
10
>>> num2
5
>>>
>>> # + -> Addition
>>> num1 + num2
15
>>>
>>> # - => Subtraction
>>>
>>> num1 - num2
5
>>>
>>> # * => Multiplication
>>>
>>> num1 * num2
50
```

```
>>> # ** -> Exponent
>>>
>>> 5 ** 2
25
>>>
>>> 3 ** 4
81
...
```

```
>>> # % => Modulus
>>>
>>> 10 % 3
1
...
```

```
>>> # / => Division
>>>
>>> num1 / num2
2.0
```

in division operator / the output will be in float value

```
>>> # // => Floor division
>>> num1 // num2
2
>>>
>>> 10 / 3
3.3333333333333335
>>>
>>> 10 // 3
3
```

only the integer quotient

when we use % symbol we get the reminder of the division

```
>>> # Assignment operators
>>>
>>> value = 100
>>>
>>> add = num1 + num2
>>> add
15
```

Compound assignment of operations

```
>>> x = 10
>>>
>>> x
10
>>>
>>> x += 1 # x = x + 1
>>>
>>> x
11
>>>
>>> x -= 6 # x = x - 6
>>>
>>> x
5
>>>
>>> x *= 10 # x = x * 10
>>>
>>> x
50
>>>
>>> x /= 20 # x = x / 20
>>> x
2.5
```

Comparison & Logical Operators

```
>>> # comparision operators
>>> # ==
>>> num1 = 21
>>> num2 = 12
>>> num1 == num2
False
>>> num3 = 21
>>> num1 == num3
True
```

```
>>> num1 = 21
>>> num2 = 12
>>> num1 = num2
>>> num1
12
>>> num2
12
```

== means is this both equal or not

```
>>> num1 = 'hi'
>>> num2 = 'Hi'
>>> num3 = 'hi'
>>> num1==num2
False
>>> num1==num3
True
```

!= means $\neq$

```
>>> # !=
>>> num1 = 21
>>> num2 = 12
>>> num3 = 21
>>> num1 != num2
True
>>> num1 != num3
False
```

and also same for strings

```
>>> # > greater than
>>> num1 = 21
>>> num2 = 12
>>> num3 = 21
>>> num1 > num2
True
>>> num2 > num3
False
>>> # < less than
>>> num1 < num2
False
>>> num2 < num1
True
```

```
>>> # >= greater than or equal to
>>> num1 =21
>>> num2 =12
>>> num3 =21
>>> num1 >= num2
True
>>> num1 >= num3
True
>>> num2 >= num3
False
>>> # <= lesser than or equal to
>>> num1 <= num2
False
>>> num2 <= num1
True
>>> num2 <= num3
True
```



```

>>> # logical operators  and,or,not
>>> \
>>> # and
>>> True and True
True
>>> True and False
False
>>> False and False
False
>>> # eg : I am Jash , i am 20
>>> True and True
True
>>> # eg : I am Jash , i am 45
>>> True and False
False
>>> # eg : I am John , i am 20
>>> False and True
False
>>> # eg : I am John , i am 45
>>> False and False
False

```

```

>>> # or
>>> True or True
True
>>> True or False
True
>>> False or True
True
>>> False or False
False

```

```

>>> # not
>>> not True
False
>>> not False
True

```



```

>>> name = 'Jash'
>>> age = 20
>>> name = 'Jash' and age >= 18
>>>
>>> name == 'Jash' and age >= 18
False
>>> name = 'Jash'
>>> age = 20
>>> name == 'Jash' and age >= 20
True
>>> name == 'John' and age >= 20
False
>>> name == 'John' and age <= 20
False
>>> name == 'John' or age >= 20
True
>>> name == 'JOhn' or age <= 20
True
>>> name == 'John' or age <= 18
False

```

Associativity & Precedence of operators

precedence = are priority in which operations are performed in python

eg : BODMAS common precedence

```

>>> # Precedence
>>>
>>> 5 + 10 * 6
65

```

precedence in logical operators ..

```
>>> name = 'Jash'
>>> age = 20
>>> name == 'Jash' or name == 'John' and age < 18
True
```

[True or Flase] and False should be false but output is true

or is not getting evaluated first from thus we can understand it is evaluating and first because of precedence ..

True or [False and False]

priority : () > and > or

```
>>> (name == 'Jash' or name == 'John') and age < 18
False
>>>
```

if paranthesis is used then those operation is done first then next one is done ..

in python multiplication and division are having same precedence ...

ASSOCIATIVITY

```
>>> 100 / 10 * 10
100.0
>>>
>>>
>>>
>>> 2 ** 1 ** 3
2
```

IF THE ORDER OR PRECEDENCE IS SAME THEN
WHICH OPERATION IS DONE FIRST IT COULD BE
LEFT TO RIGHT OR RIGHT TO LEFT THIS IS
COMMONLY CALLED AS ASSOCIATIVITY ..

4	**	Exponentiation	Right to left
5	+x, -x, ~x	Positive, negative, bitwise NOT	Right to left
6	*, @, /, //, %	Multiplication, matrix, division, floor division, remainder	Left to right

SAME PRESEDENCE HENCE ASSOCIATIVITY DEFINES WHICH OPERATION FIRST IN MEANS OF DIRECTION LEFT OR RIGHT STARTING

CLASSIFICATION OF OPERATORS BASED ON NUMBER OF OPERANDS

```
# Unary
-10
-10

not True
False

# Binary
10 - 5
5

True or False
True
```

Printing to console - the print() function

```
name = "John"
print(name)
John
age = 20
print(age)
20
```

```
>> print(name, age)
John 20

>>
>> print(name, age)
John 20
```

sep is a separator function ...

```
print(10, 20, 30, 40, sep=" ")
10 20 30 40
print(10, 20, 30, 40, sep=",")
10,20,30,40
print(10, 20, 30, 49, sep="?")
10?20?30?49
```

```
print("python")
python
print("python')
```

SyntaxError: unterminated string literal
detected at line 1)

starting with double quote need to end with same quote which started or it will be a syntax error\

```
num1 = 100
num2 = 200
result = num1 + num2
print(num1, num2, result)
100 200 300
```

```
print("Name: John, Age: 20")
```

```
Name: John, Age: 20
```

```
print("Name: John Age: 20")
```

```
Name: John Age: 20
```

Taking input from user - the input() function

```
>>> first_name = input()
```

```
John
```

```
>>> print(first_name)
```

```
John
```

```
>>>
```

```
>>> age = input()
```

```
25
```

```
>>> print(age)
```

```
25
```

input() gives option for user to give input the user who is entering the details can't see the first_name or age only the code person can see it so we need to make sure to give prompt

```
first_name = input("Enter your name")
```

```
Enter your nameJill
```

```
print(first_name)
```

```
Jill
```

```
age = input("How old are you? ")
```

```
How old are you? 30
```

```
print(age)
```

```
30
```

very important thing to know that any input fetching from the user it is always in the format of strings not in the format of numbers ...

```
num1 = input("enter a number :  ")
```

```
enter a number :  10
```

```
num2 = input("enter a number: ")
```

```
enter a number: 15
```

```
result=num1+num2
```

```
print(result)
```

```
1015
```

see the above example if those are numbers out put would be the 25 but it read as string so output as 1015

string concatenation = string addition

```
type(num1)
<class 'str'>
type(num2)
<class 'str'>
num1 + num2
'1015'
```

```

int(num1)
10
int(num2)
15

result = int(num1) + int(num2)
print(result)
25

```

just convert string into integers (numbers)

```

year = input("What is the current year? ")
What is the current year? 2020
age = input("What is your age? ")
What is your age? 45

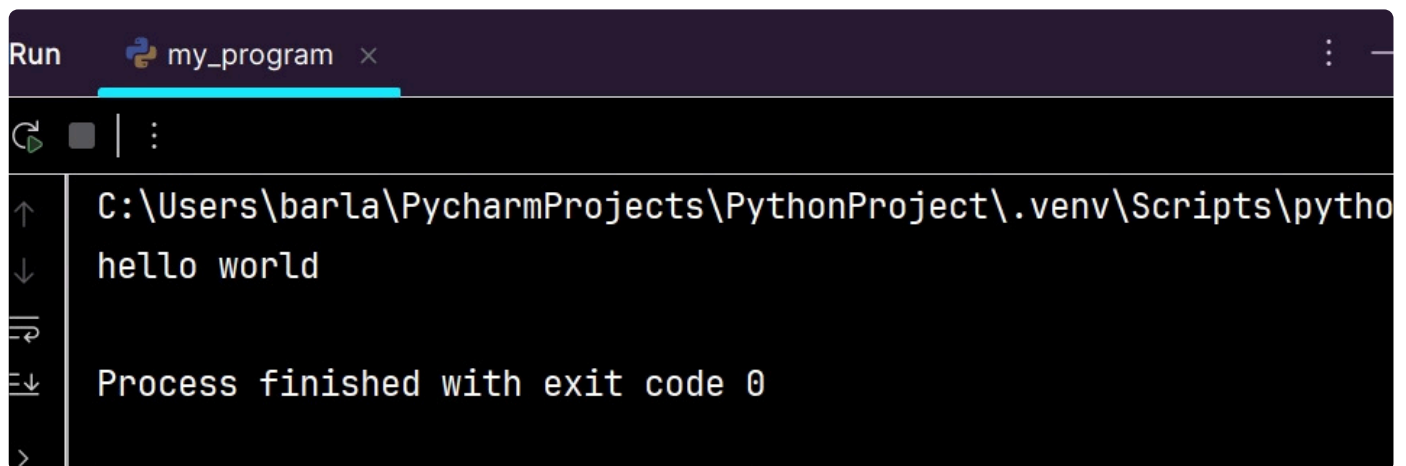
# You were born in the year 1975

year - age
Traceback (most recent call last):
  File "<pyshell#38>", line 1, in <module>
    year - age
TypeError: unsupported operand type(s) for -: 'str' and 'str'
int(year) - int(age)
1975
born_year = int(year) - int(age)
print("You were born in the year", born_year)
You were born in the year 1975

```

**the reason why the type error because we are trying to string - string
so we converted the data type and then we subtracted which is
correct**

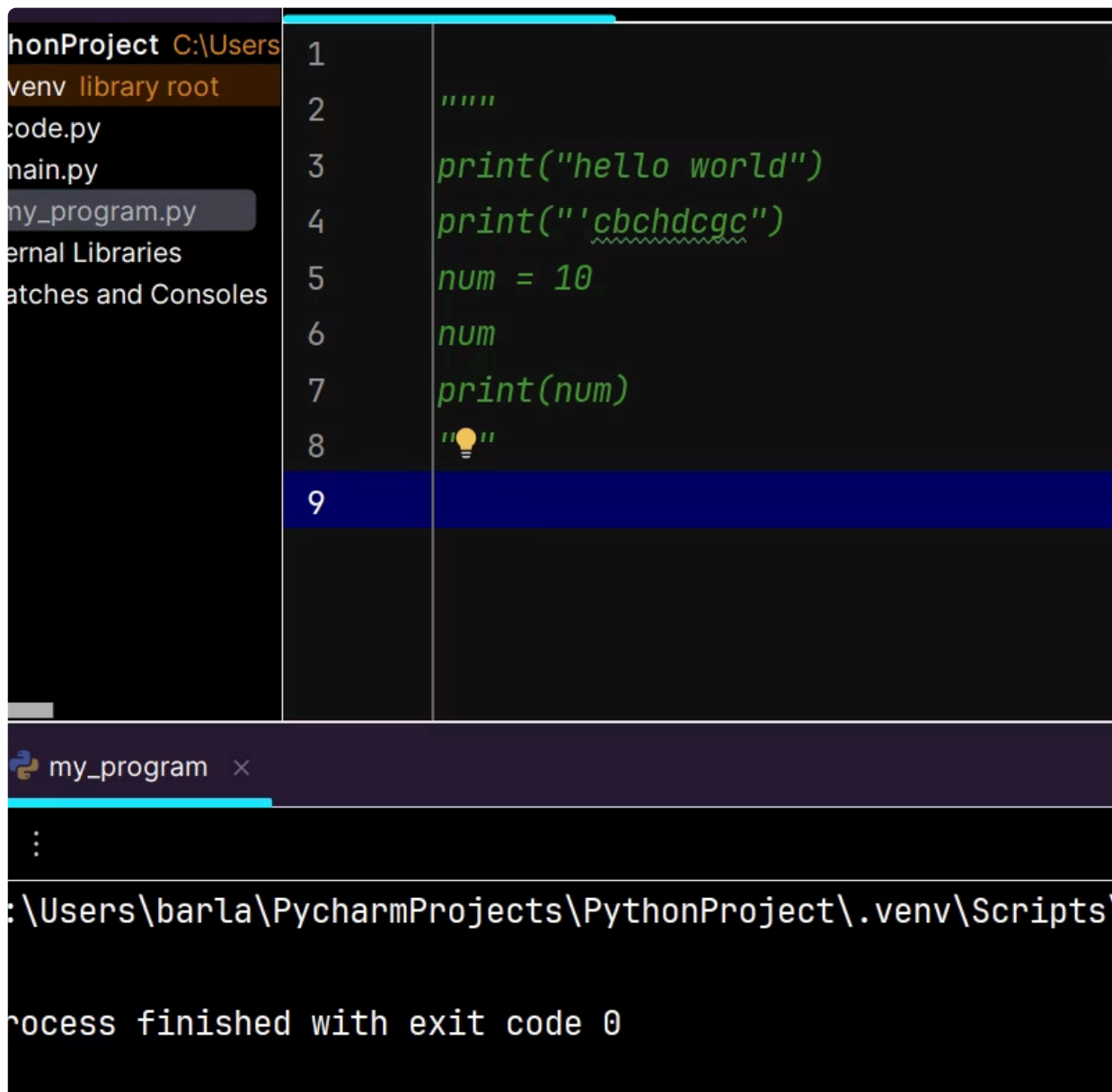
pycharm IDLE



The image shows the 'Run' window in PyCharm. The title bar says 'Run' and 'my_program'. The output area shows the command prompt path 'C:\Users\barla\PycharmProjects\PythonProject\.venv\Scripts\python' followed by 'hello world' and 'Process finished with exit code 0'.

```
Run my_program x
C:\Users\barla\PycharmProjects\PythonProject\.venv\Scripts\python
hello world
Process finished with exit code 0
```

exit code 0 means code run without any error



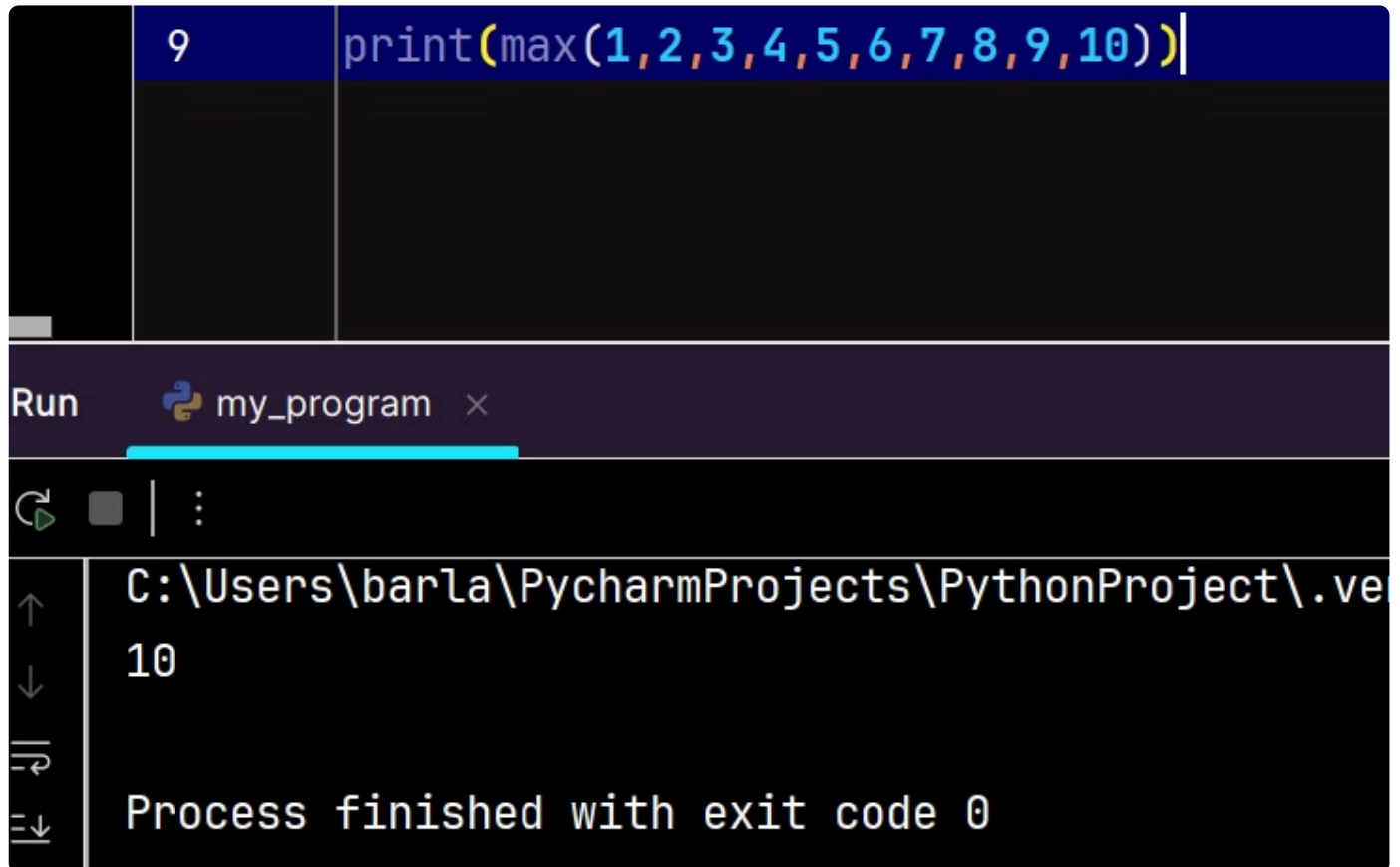
The image shows the PyCharm editor with a file named 'my_program.py' open. The code in the editor is as follows:

```
1
2 """
3 print("hello world")
4 print("'cbchdcgc'")
5 num = 10
6 num
7 print(num)
8 """
9
```

Below the editor is the 'Run' window, which shows the same output as the first image: 'C:\Users\barla\PycharmProjects\PythonProject\.venv\Scripts\python', 'hello world', and 'Process finished with exit code 0'.

using """ """ just becomes numb and they wont run and you cant see its output in output terminal

numeric functions :



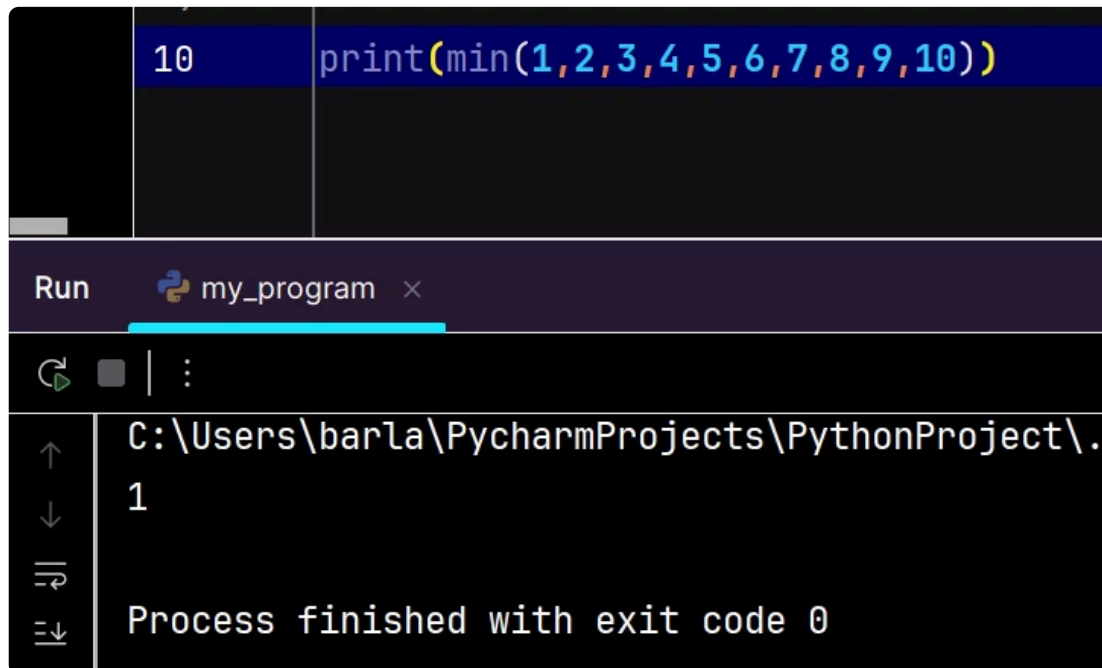
The screenshot shows a Python IDE with a code editor and a run console. The code editor contains the following code:

```
9 print(max(1,2,3,4,5,6,7,8,9,10))
```

The run console shows the output of the code:

```
Run my_program x  
C:\Users\barla\PycharmProjects\PythonProject\.venv  
10  
Process finished with exit code 0
```

max function identifies maximum number in the integers



The screenshot shows a Python IDE with a code editor and a run console. The code editor contains the following code:

```
10 print(min(1,2,3,4,5,6,7,8,9,10))
```

The run console shows the output of the code:

```
Run my_program x  
C:\Users\barla\PycharmProjects\PythonProject\.venv  
1  
Process finished with exit code 0
```

min function identifies minimum number in the integers

```
5 #print(abs(-2020))
6
7 print(pow(2,3))
```

abs absolute value -2020 absolute value is 2020

pow is power 2 power 3 which is 8

Coding exercises